The Tic-Tac-Toe Robot

Department of Mechanical and Mechatronics Engineering

A Report Prepared For:

The University of Waterloo

MTE 100

Prepared By:

Group 27

Maxim Beauchemin

Chris Dietcher

Andrew Jang

Presley Prince

December 2, 2025

December 2, 2025

MTE 100 and MTE 121 Teaching Teams
Department of Mechanical and Mechatronics Engineering
University of Waterloo
200 University Ave W.
Waterloo, Ontario, N2L 3G1

Dear Teaching Teams,

As a team, we would like to express our deepest gratitude to the entire teaching team. Without you we would not have come up with this project which, while having had several very late nights and a few clawed-out hairs, we are incredibly proud to have created.

Instruction from all our professors was essential to learn the theory to build this robot. From Professor Bedi's fun lectures of components and the design cycle to Abdullah's EGAD labs, and Professor Mostafa's programming PowerPoints. The project design, 3D-printed parts, and C++ programming would have all been impossible without their foundational teaching. Ryan Consell's advice proceeding the formal presentation was key in many parts of our design.

Additionally, the MTE 100 tutors were key in the project execution. Their continuous feedback throughout the project and near-constant availability made many of our largest issues into just a quick visit to their office.

Lastly, we would like to thank the amazing co-op students at WatIMake who taught us the basics of 3D printing.

This report was written entirely by Group 27. Apart from acknowledgements listed above and references, the program and robot were created entirely by Group 27 as well.

Best Regards,
Maxim Beauchemin
Chris Dietcher
Andrew Jang
Presley Prince

Table of Contents

List of Figures iv

List of Tables v

Summary vi

1.0 Introduction 1

2.0 Scope 2

2.1 Functionality 2

2.2 Measurements and Readings 4

2.3 Interaction with the Environment 5

2.4 Task Completion 6

2.5 Changes to Scope 6

3.0 Constraints and Criteria 8

3.1 Final Constraints and Implemented Changes 8

3.2 Final Criteria and Implemented Changes 8

3.3 Importance of Each Criteria and Constraint 9

4.0 Mechanical Design and Implementation 10

4.1 Mechanical Design Overview 10

4.2 X-Axis Motor and Crane Base 11

4.3 Y-Axis Motion and Electronics Mounting 12

4.4 The Claw 13

4.5 Z-Axis Motion 14

4.6 Support Wheel 15

4.7 Board design 16

5.0 Software Design and Implementation 17

5.1 Intro to the Program 17

5.2 Task List 17

5.3 The Functions 18
5.4 Data Storage 22
5.5 Decisions and Trade-offs 23
5.6 Testing Procedures 24
5.7 Significant Problems 25
6.0 Verification 26
6.1 Fulfilled Constraints 26
6.2 Failed Constraints 26
7.0 Project Plan 28
7.1 Tasks Splitting 29
7.2 Revisions to Project Plan 29
7.3 Project Plan vs Actual Timing 30
8.0 Conclusions 31
8.1 Successes 31
8.2 Failures 31
9.0 Recommendations 32
9.1 Mechanical Recommendations 32
9.2 Software Recommendations 33
References 34
Appendix A – C++ Code 1

List of Figures

Figure 1. Tic-tac-toe board and pieces 2
Figure 2. Starting position of robot 3
Figure 3. Robot scanning the board 4
Figure 4. Motors and their axes 6
Figure 5. Distance sensor location after redesign 7
Figure 6. Preliminary design of Tic Tac Toe Robot 10
Figure 7. Final robot mechanical design 11
Figure 8. Preliminary x-axis motion design 11
Figure 9. Final x-axis design 12
Figure 10. Initial vs final Y-axis design 13
Figure 11. Claw design 14
Figure 12. Final Claw design 14
Figure 13. Z-axis motor 15
Figure 14. Support wheel and its path guides. 16
Figure 15. Final board design 16
Figure 16. Flowchart for main 18
Figure 17. Flowchart for the waitOrEndProgram function. 19
Figure 18. Flowchart for moveToXY. 19
Figure 19. Flowchart for moveToZ 20
Figure 20. Flowchart for clawOn 20
Figure 21. Flowchart for checkWin. 21
Figure 22. Flowchart for subOptimalSquare 21
Figure 23. Flowchart for scanBoard 21
Figure 24. Flowchart for makePlay. 22
Figure 25. Flowchart for resetPosition 22
Figure 26. Board representation including array access indices and user location. 23
Figure 27. A description of the project plan after the formal presentation. Taken from the formal presentation slide show. 28
Figure 28. Piece was placed off-centered by robot 32

List of Tables

Table 1. A table of all X and Y positions for the motors for each square of the board. The position is given in degrees and obtained from motor encoders through testing. The array indices are visualized in Figure 26. 24

Table 2. A table of distance sensor readings corresponding to the number of pieces left in the stack. The values assume that all pieces are left in the stack or on the board. The values were obtained through distance sensor reading and refined in testing. 25

Summary

The MTE100 robot project for Chris, Maxim, Andrew and Presley was an autonomous tic-tac-toe robot. This robot was designed to solve the problem of one of the team members being too busy at university to play his cousin in tic-tac-toe.

The project started following the design cycle and discussing the constraints and criteria for robot design. After need synthesis was complete through analysis of criteria and constraints, preliminary designs were drafted for the robot's function. Designs for given parts were selected and then manufactured into a functional robot. There were many iterations of redesign adjusting both small and large details about the robot's design. Next, the robot was programed to play a game of tic-tac-toe, using a color sensor to scan the board and a three-axis motor system with a claw to move pieces.

Various methods of verification were used to test the robot's functionality and adjust preprogramed encoder values. During the validation phase of the design cycle problems started to arise in the robot's functionality. Namely, the Z-axis motor was too weak to carry the load of the claw its full vertical distance and encoders inaccuracies made piece pick up difficult for the robot. However, in the final demo the robot was able to perform tic-tac-toe moves from various starting positions but could not play a full game due to the previously described errors.

It is concluded, for future designs of a tic-tac-toe robot to redesign the Z-axis motor with potentially a gearing system to reduce torque, use lead screws to increase motor encoder accuracy, and consider using the Minimax algorithm to further optimize the robot's gameplay.

1.0 Introduction

The MTE100 project is a first-year design project in which students find a problem and design a robotics solution to solve the problem. The criteria for this project were that it must solve a problem, use at least 3 motors and 4 inputs, and contain at least six functions in the robot's code. This design project report will focus on the development of an autonomous tic-tac-toe robot.

This MTE100 robot project was designed to solve the problem of one of the group members not being able to beat his cousin at tic-tac-toe. Chris has a long-standing tic-tac-toe rivalry with his 8-year-old cousin and cannot beat him since he is too busy studying at Waterloo. So, the group devised a plan to make an autonomous tic-tac-toe robot to beat Chris's cousin while he is away. The goal for this robot was to design a simple to use, autonomous tic-tac-toe robot capable of playing full games of tic-tac-toe with little user assistance.

Tic-tac-toe is a simple game involving players taking turns placing X's or O's on a 3x3 grid. The goal is to get three X's or O's in a row to win. The robot is designed to be able to scan any tic-tac-toe board and play a specific move to bring it closer to winning or tying the game. The robot is also designed to play against a player having a button to press signifying it is the robot's turn. This design allows it in theory to play whole games against another player, itself or from any given valid position.

This robot was developed using Vex IQ electronics and parts, Tetrix parts and various 3d printed elements. Vex IQ is an intuitive design framework for assembling basic robotics systems using plastic, snap-together pieces [1]. Additionally, Vex IQ has very inaccurate sensors that were a major source of troubleshooting for later design iterations of the robot. Tetrix parts were used for main stabilization of robot design since the aluminum extrusions were more rigid than plastic Vex IQ parts. Lastly, custom parts were made to fit specific functions of the robot that would be hard to model using Vex IQ or Tetrix parts. The primary use for 3D printer was for complex parts like the Claw and Board which will be talked about in further detail in 4.0 Mechanical Design and Implementation.

2.0 Scope

2.1 Functionality

The robot autonomously played tic-tac-toe by moving physical pieces from a stack onto a board. The pieces and board are shown in Figure 1. To play, the robot recognized when it was its turn, scanned the board, and placed a piece on the board.

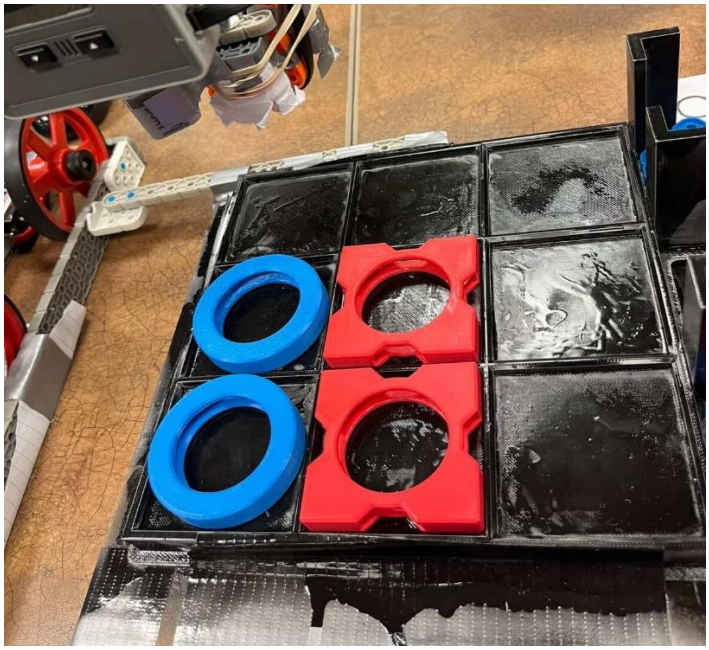


Figure 1. Tic-tac-toe board and pieces

During regular operation, the robot would wait for the user to make their move and press the touchLED. The press informed the robot that it was its turn. The robot would then zero itself by moving to its starting position. This starting position was located at the top-left corner of the board relative to the user. In this position, the claw assembly was also raised to its highest possible elevation. The starting position is shown in Figure 2.

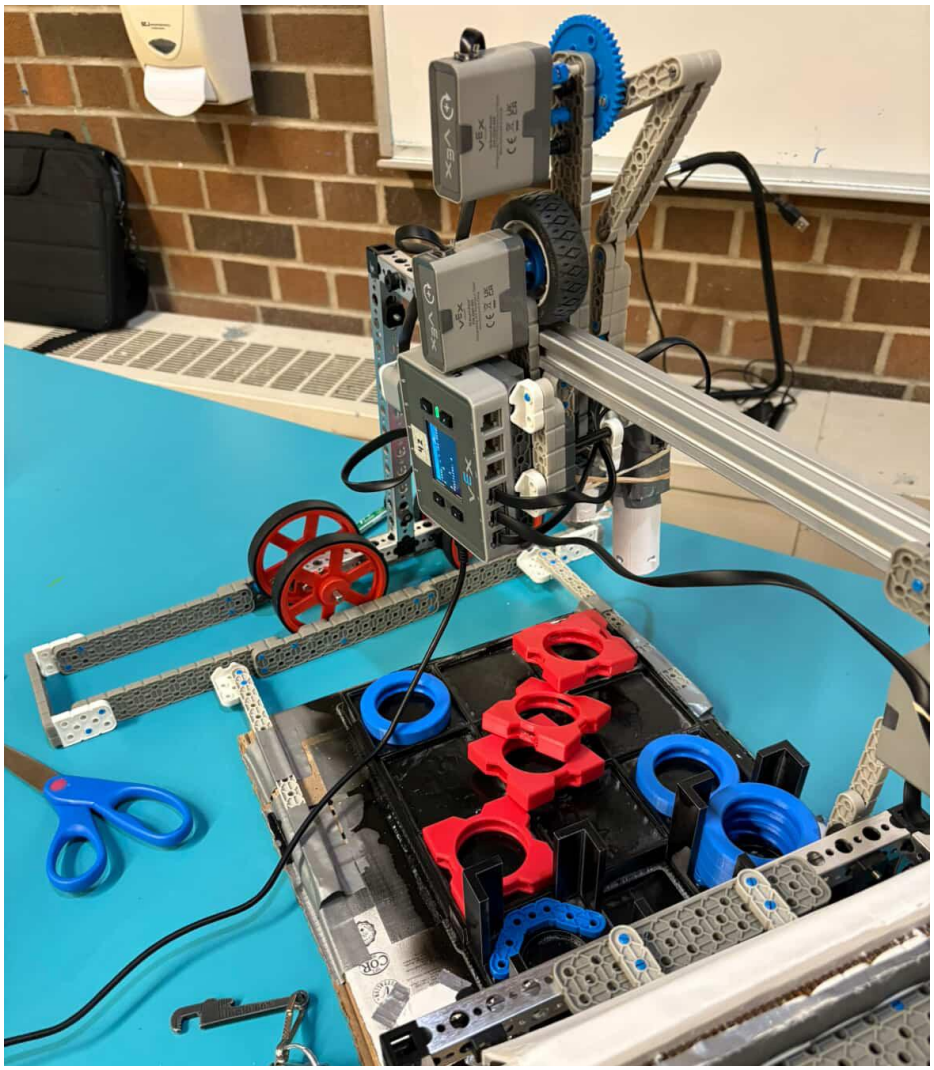


Figure 2. Starting position of robot

Once the robot reached its starting position, it would then proceed to scan the board. Scanning the board allowed the robot to determine what pieces have been placed, where said pieces were, and what piece it should place and where it should place it. Scanning the board also enabled the robot to see if the game was over, or if any cheating has been done by the user. For this project, cheating was defined as having an invalid number of pieces on the board. Since X always goes first, the number of X-pieces must have been one greater or equal to the number of O-pieces on board. To scan the board, the robot lowered the claw assembly to its scan height and then moved the color sensor (located on the claw assembly) over each space on the board. The scan height was the lowest height the claw could go without hitting any pieces placed on the board. This is illustrated in Figure 3. After completing the scan, the robot returned to its starting position.

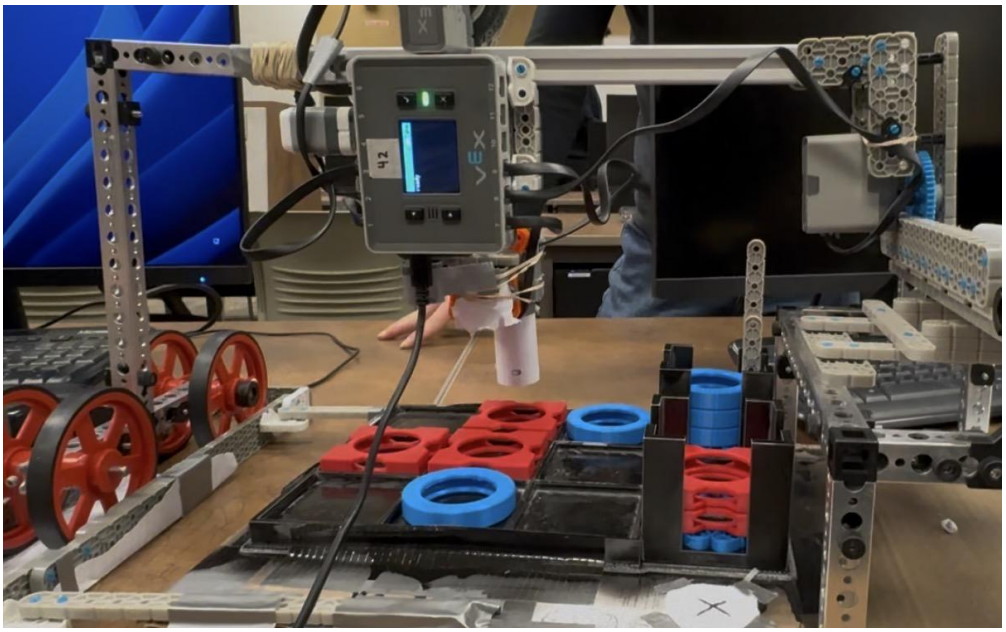


Figure 3. Robot scanning the board

The robot's next task was to pick up a piece and place it on the board. This task was only completed if the game was not over and if cheating was not detected. First, the robot would move to the correct stack of pieces based on the scan of the board. The claw assembly would then be released and allowed to fall to its lowest height. The claw would then be raised to the height of the top piece in the stack. The claw was then extended into the holes of the piece. The claw assembly would then return to its starting position for placement. After returning to the starting position, the robot moved the claw assembly to the space it determined would result in the best move. The claw is then lowered and then retracted, allowing the piece to fall into place on the board.

The robot then returned to the starting position and waited for the user to complete their next turn.

2.2 Measurements and Readings

Throughout the robot's operation, the robot kept track of its position, differentiating between the different pieces on the board, and whether it was its turn. The robot also used the inputs to decide when to end the program. The robot received readings from the touchLED, motor encoders, color sensor, and distance sensor.

The touchLED acted as a method of communication between the user and the robot. Pressing the touchLED informed the robot that the user had completed their turn and that it was the robot's turn. The touchLED also acted as the shutdown mechanism for the robot. While the robot is waiting for the user, holding down the touchLED for three seconds or more would have shut down the robot.

The motor encoders were used to keep track of the position of the claw assembly throughout the entire operation of the robot. This includes the position in the x-axis, y-axis, and z-axis. Keeping track of the claw's position was important as the robot had to move to many different locations in three-dimensional space. These locations included each space on the board, the two stacks of pieces, and each of the five possible heights in each stack. Only the encoder values of three of the four motors were used to measure position; the fourth motor did not control the motion of the claw assembly. The motor encoder values were measured in degrees.

The color sensor was used to differentiate between X-pieces, O-pieces, and empty spaces. The X-pieces were red, the O-pieces were blue, and the board was black. When scanning space on the board, the color sensor took 20 measurements and used the lowest hue value. See [Appendix A, line 564] for more information on the color differentiation process.

The distance sensor was used to keep track of the height of the claw when the robot was picking up pieces from the stacks. The distance sensor was used in place of the encoder for the z-axis motor because the distance sensor was believed to be more accurate. Increased accuracy was required when picking up pieces due to the small size of the pieces.

2.3 Interaction with the Environment

Interaction with the environment was accomplished using the robot's four motors. Three of the motors were used to control the position of the claw assembly. Each of these motors was responsible for movement along one of the three main axes. One motor controlled x-axis motion; another motor controlled y-axis motion; a third motor controlled z-axis motion. The motors and their respective axes are shown in Figure 4. These three motors allowed the robot to reach each of the nine spaces on the board, the two stacks, and each of the five heights in both stacks. These motors also allowed the robot to properly position the color sensor for reading the board. The fourth motor was located on the claw and controlled its extension and retraction. This motor enabled the robot to pick up and put down pieces.

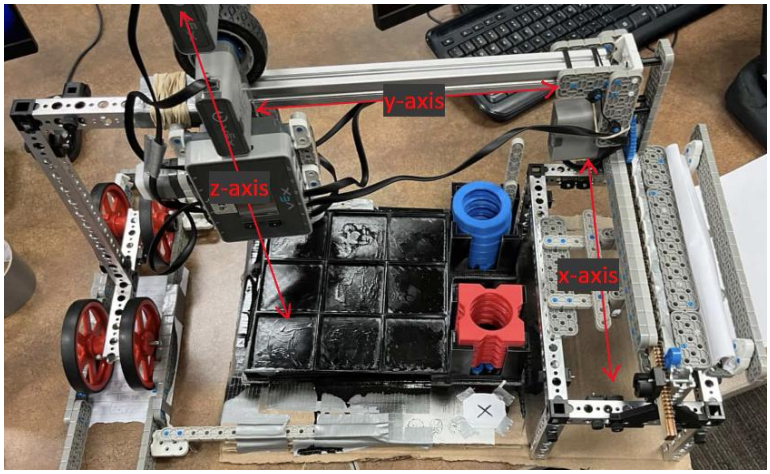


Figure 4. Motors and their axes

2.4 Task Completion

There were four main tasks the robot accomplished during regular operation: waiting for the user, returning to the starting position, scanning the board, and placing pieces. The robot recognized the completion of each of these tasks in different ways. Waiting for the user was completed when the touchLED was pressed. Returning to the starting position was completed by spinning the motors in the direction of the starting position and setting a timer. This task was completed when the timer reached the reset time of four seconds for the x-direction and y-direction and three seconds for the z-direction. Scanning the board was completed after the robot had hue values for all nine spaces on the board. Placing a piece was completed after the robot moved to the location it wanted to place the piece (which was monitored with the motor encoders) and retracted the claw.

The robot was designed to keep playing until the game was won or tied, or the robot detected cheating. Once one of these scenarios was encountered, the robot would wait for the user to reset or alter the board to correct the issue. As a result, the robot could play an unlimited number of Tic-Tac-Toe games. The program only shuts down when the user holds the touchLED for three seconds between turns.

2.5 Changes to Scope

Throughout the course of the project many alterations were made to the scope of the project to improve the functionality of the robot.

One of the first changes made was the shutdown procedure. The shutdown procedure was originally intended to be double tapping the touchLED instead of holding the touchLED. This change was

made to increase the simplicity of the code. In addition, holding the touchLED felt more intuitive during testing. Another change to the shutdown procedure was only allowing it to occur between turns. The change was implemented for two reasons. First, it made the robot logic simpler to implement as the robot would not be constantly checking the touchLED. Second, when the program stopped, the claw dropped down and could hit the stacks, board, or pieces. While between turns, the robot was in the starting position. In this position, there were no objects that the claw could hit when it fell. Only allowing the robot to shut down between turns prevented any damage to the claw mechanism.

Another change to the functionality of the robot is the frequent zeroing of the robot. Initially, the robot was planned to only return to its starting position at the start of its turn. While this approach worked in principle, the inaccuracy of the motor encoders caused an accumulative positional error. The frequent returns to the starting position made the robot more accurate and prevented any positional errors from accumulating.

Another change to the robot was how the distance sensor was used and the piece pick up process. Originally the distance sensor was going to be used to determine the height of the pieces in the stack and the claw would be lowered to said height. However, this approach had two main issues. The size of the claw made it difficult to position the distance sensor in such a way that the robot could measure the distance to the top of the stack and grab the topmost piece. This issue is illustrated in Figure 5. The other issue was lowering the claw to the proper height was inaccurate and resulted in the robot grabbing the wrong piece. This is why the distance sensor was changed to measure the height of the claw relative to the board, and the claw was dropped and then pulled back up to the desired height. The distance sensor was given the necessary space and pulling the claw back up was more accurate than lowering the claw.

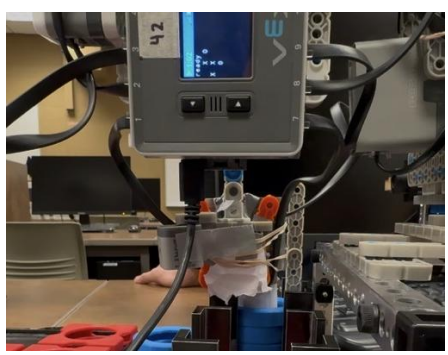


Figure 5. Distance sensor location after redesign

3.0 Constraints and Criteria

3.1 Final Constraints and Implemented Changes

Many of the constraints concerning robot design originated from restrictions on electronic components. The initial constraint was short wire lengths, as there was only one 400mm wire provided. Additionally, the number of motors was limited to four, so the motion of the claw in the x and y direction was not as reliable as it could have been. Due to these factors, the size of the board and its pieces were limited, since three of four motors, which are connected to the brain through a wire, had to reach all four corners of the board as shown in Figure 4. In addition, the brain's location skewed the movement of the aluminum extrusion in the x-direction, due to the high mass of the brain's battery. Furthermore, VEX IQ motors were found to be imprecise during testing, and their strength depleted with lowering battery levels. Since the motor encoder values were battery-level specific and the rate at which the battery depleted was high, attempts at finding the perfect measurements to reach the different heights needed were difficult. The optical sensor was found to be imprecise as well. Red was sometimes perceived as black and vice versa. The latter two constraints were updated to be part of the constraints list halfway into building the robot.

Structurally, there were not enough fasteners in the kit provided to hold everything together. 4 threaded plates were bought to connect the aluminum extrusion to the wheel mechanism, but there were many other locations at which the robot could have been more secure. Unfortunately, money was a constraint as well, so it was not possible to buy enough fasteners to secure everything properly. Additionally, there was an issue with 3D printer resolution, as the printed X pieces were found to be too short compared to the specifications of the CAD model. Some pieces were also printed with rugged textures. Finally, smooth motion with VEX IQ beams was difficult due to the ridges present in their design. It was very easy for gears to get caught within these ridges.

3.2 Final Criteria and Implemented Changes

Some of the criteria that are to be discussed were derived from the constraints. These are mentioned in this section but will be further addressed in 6.0 Verification. To account for the wire-length constraint, it was decided that the brain would be placed onto the y-axis motion mechanism. This was not an initial criterion; originally, the brain was supposed to be located near the linear rack system. This was deduced from the criterion that for electronic components to work, they must be powered by the brain using wires. Additionally, due to low motor precision, low 3D printer resolution, and claw-

piece friction, there must be a reasonable amount of clearance in the fit between the claw and the pieces, and the pieces and the board. Moreover, due to the many constraints on x-axis motion, the mechanism responsible for it must have been reinforced.

It is required that the player be given an opportunity to play against the robot after its turn, but it is up to the player if they wish to play. The robot must have made a turn after a press of the touchLED, even if the player has not played. The claw must return to zero position after its move so the user can make their move without the robot blocking them. The claw must also be able to pick up and place pieces onto the correct square on the board. Subsequently, the claw must be able to move to all points on the board (x-y direction) and vertically (z direction) so it can reach all possible pickup and release positions. The claw should also be able to convert rotational motion to linear motion, since it must have some difference in positioning, i.e. the location of the protrusions, to pick up and release pieces.

To increase reliability, it was also decided that the aluminum extrusion must be supported on both sides due to the amount of weight that it carries, as shown in 2.2 Measurements and Readings Figure 3.The initial design was to make it a cantilever, but that was found to be too unrealistic after considering all possible locations of the brain and motors. The design of the claw led to the design of the pieces, which should have circular holes so they could be picked up from the inside and be of distinct colors for easy optical sensor and human distinction. Therefore, the optical sensor was decided as the sensor responsible for reading information from the board. The distance sensor should have also been used to determine the height of the stack of X and O pieces, so that the robot could determine the next height at which a piece needs to be picked up.

3.3 Importance of Each Criteria and Constraint

The limited number of motors and short wire lengths were the main constraints behind the size and positioning of the components, especially the brain, motors, and sensors of our robot. Given the restrictions on buying external electronic components and the low accuracy of the existing ones, the group was forced to design around these constraints. The criteria related to how tic-tac-toe is played, such as checking for win conditions, how the robot must wait its turn, and positioning of the claw for user convenience heavily inspired the software design of the robot. The rest of the criteria and constraints did not guide design as much, either because they were deducted from existing constraints or criteria, laws of nature, or because the group was aware of it since the very beginning of the project.

4.0 Mechanical Design and Implementation

4.1 Mechanical Design Overview

Throughout the course of the project, the overall and part design of the robot has been through many iterations of the design cycle. Initially, the first drafts of the robot depicted it as a primarily crane-like structure overtop of the board, with the main components contained inside the crane, see Figure 6.

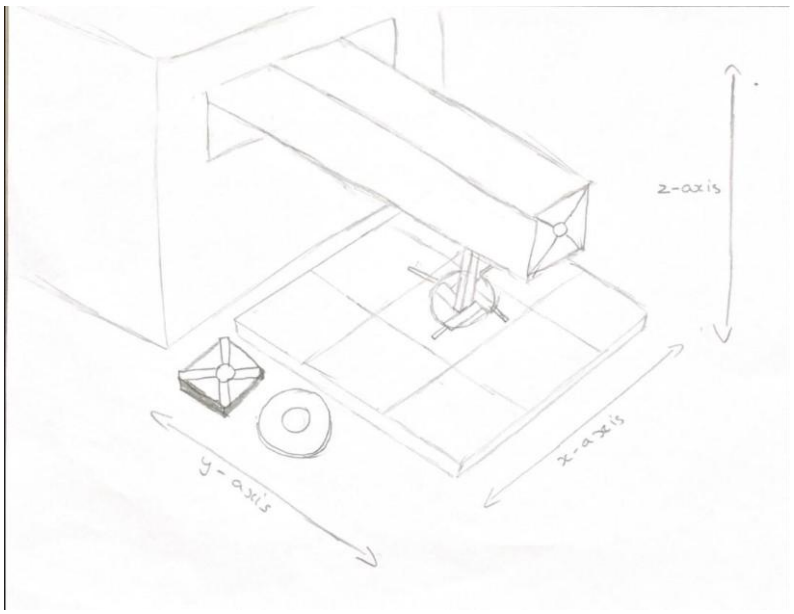


Figure 6. Preliminary design of Tic Tac Toe Robot

This design contained the same three axis of motion principle used in the final robot design although it had some critical flaws. Namely, the wire length constraint prevented the storage of the robot brain and battery inside the crane and the high torque from battery weight, as mentioned in 3.1 Final Constraints and Implemented Changes, disabled motion on the x axis. To fix these issues, the robot design was altered to being supported on both sides with the brain mounted on the claw, as further 3.2 Final Criteria and Implemented Changes. Other edits of individual parts in the design were also made and will be elaborated on in further sections of Mechanical Design. The final system after adding beam support and the brain on the claw can be seen in Figure 7. This design was decided upon since it met the criteria and functioned within the constraints described in 3.0 Constrains and Criteria. Namely, the constraint of short cable length was the most limiting factor in the design requiring a tradeoff for a smaller scale robot. This also hindered the tolerance for piece drop off on the board due to the board size shrinking.

Figure 7. Final robot mechanical design

4.2 X-Axis Motor and Crane Base

The crane base is a stable structure that supports the overall robot weight during operation. It was made using a rigid aluminum hull to guarantee structural stability and Lego to support and improve x-axis motor functionality. Additionally, the crane base is secured to the cardboard with cut grooves to ensure that the board is locked in a position relative to the crane, improving motor encoder accuracy. These details can be seen above in Figure 7.

The X-axis motor originally was designed to incorporate a chain-based system for motion that can be seen in Figure 8. Although there were a few critical flaws with this design: the wire length not being able to reach sprocket from new brain length, mounting difficulties with Vex/Tetrix parts not meshing well, and design difficulty as no member had previous experience with developing chain and sprocket systems. Thus, a new design was implemented using a rack and pinion system to account for the issues of wire length and lack of group knowledge.

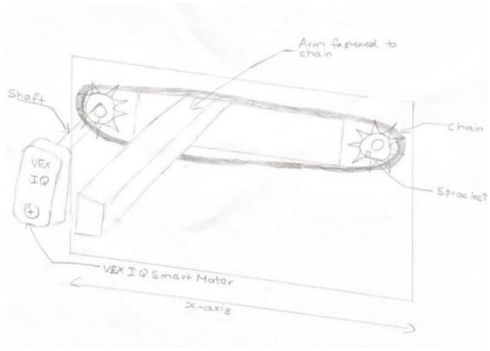


Figure 8. Preliminary x-axis motion design

The rack and pinion design was then implemented. There are a few details that had to be redesigned about this rack and pinion system before it was ready for operation. First, a firm row of corner connectors acting as a stabilizer was put in place to ensure the pinion would not slip off. Second, an axle support was added on the far side of the motor to decrease the load on the motor. Lastly, duct tape was added to prevent the axle stabilizer from getting caught on the pinion guide rails. All of these solutions are further addressed in 6.1 Fulfilling Constraints. After these mechanical redesigns, the final X-axis motion was complete as can be seen in Figure 9.

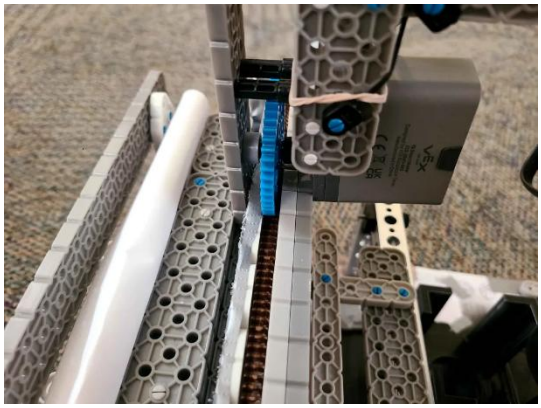


Figure 9. Final x-axis design

4.3 Y-Axis Motion and Electronics Mounting

The y-axis motion system remained mostly the same throughout the design process of the robot. Initially in the ideation phase of development, there was some debate on whether to use a rack and pinion system as well, or just a wheel on the aluminum extrusion. In the end it was decided to use the wheel system for overall simplicity, outweighing the slight precision gain of a rack and pinion system.

For the mechanical redesign of the Y-axis, the only difference between the rough drafts in ideation phase and the final design was where the wheel was supported. In the original design the wheel moved inside the Aluminum extrusion using the extrusion itself to hole it in place. In the final design however, the wheel rolled on top of the extrusion, as buying an aluminum extrusion with a groove did not meet the price range constraint. Thus, the wheel was instead supported by wrapping around the extrusion with VEX parts. These design changes can be seen in Figure 10.

As mentioned in Section 3.2, it was decided that the brain would be mounted directly onto the Y-axis motion mechanism. This would enable the robot to function withing the wire length constraint. Additionally, for the reason of the brain being on the y-axis motor, the touch sensor was also mounted on the Y-axis apparatus, to again meet wire length constraint.

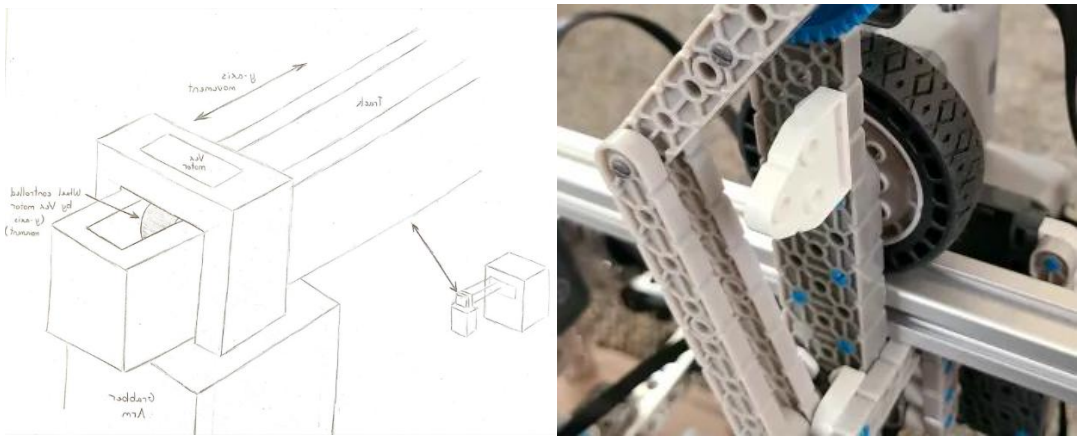


Figure 10. Initial vs final Y-axis design

4.4 The Claw

There were a few preliminary ideas that were looked at before selecting the final claw mechanism. The main three were a clamp system, double rack and pinion and a rotary pin system, the latter being the final design. When analyzing the different ideas, it was decided that rack and pinion was unnecessarily complex and that a clamp system would be difficult to calibrate for lifting singular pieces out of a stack. This resulted in the decision to use the rotary pin system seen in Figure 11. This system works by converting the rotational motion of the axle into linear motion using guide holes in the shell of the claw.

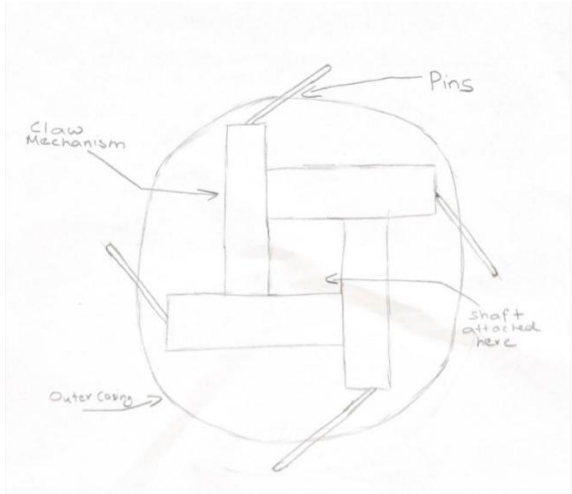


Figure 11. Claw design

This design remained the same in the final product as it was in the design document. However, an aspect of the claw design that did change was the location of the sensors. Two of our criteria were that the robot would have to be able to read the color of the pieces on the board and use the distance sensor to determine how far down the Z-axis motor would have to go. For these criteria to be met there were many iterations of mechanical redesign. These reiterations, and their respective solutions, are mentioned in Section 6.1. The final design of the claw can be seen in Figure 12.

Figure 12. Final Claw design

4.5 Z-Axis Motion

The Z-axis motion was originally planned to utilize a rack and pinion system to lift the claw up and down to required locations. However, in the final design, this was altered to a gear attached to a

freely rotating rod that would convert rotational motion to linear motion with guide rails on a third rod (see Figure 13 for more detail). The tradeoffs for this design were simplicity for precision. This new design was much easier to manufacture as it did not require any custom parts. Although it had a major drawback that the up and down motion was no longer linear as it would be with a rack and pinion system. This design also caused complications in testing causing multiple iterations of redesigning due to the ridges on the piece getting caught on guide rails. To fix this round Vex parts were used instead of ridged squares as they had less of a tendency to get caught as the claw moved up and down.

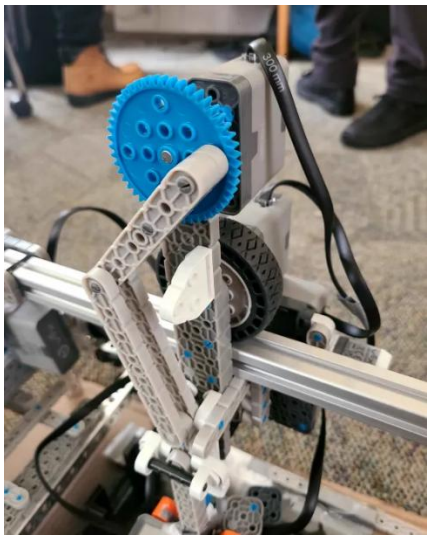


Figure 13. Z-axis motor

4.6 Support Wheel

The support wheel was added when the design was converted from a crane system to a double support system. When implementing this design decision there was a tradeoff between high torque on x axis motor and crane support and X-axis drift. Since the second stabilizer was made with a wheel protruding from the far side of the robot, there was a lot of drift with the actual motion being generated far away from the wheel. This was a large problem as it severely interfered with the preprogramed encoder values due to variable drift on the far side. However, this design was still decided upon due to no feasible simple solution to the torque problem in the original design could be found. Due to the variable nature of drift in the support wheel many improvements were made throughout the construction process. The largest one was implementing support rails for the wheels to steer their direction making drift more predictable, as further addressed in 6.1 Fulfilled Constrains. This can be seen in Figure 14.

Figure 14. Support wheel and its path guides.

4.7 Board design

The board design is a crucial part of the design that initially was planned to just be a simple 3x3 grid as seen previously in 4.1 Mechanical Design Overview Figure 6. Although as development of the robot continued, the board design became much more essential for the function of the robot. The primary change to the board was to add two stack holders for the X and O pieces. This change was designed and implemented to restrict the motion of pieces in the stack enabling the claw to have more room for error if it bumped into lower pieces. The design for these stacks also takes into consideration that the claw pins are raised a bit from the claw shell. To account for this, a circle cut out enables the claw to extend low enough and pick up the bottom piece. The second change was adding chamfers to all walls on the board. This gave the robot a wider tolerance range for depositing each given piece. Lastly, the board was painted black to reduce the reflection of light into the color senser, overall improving its performance. Figure 15 shows the final board design after various design alterations.

Figure 15. Final board design

5.0 Software Design and Implementation

The written program can be found in Appendix A. This code has been slightly modified from demonstration day to improve readability, but its functionality is unchanged.

5.1 Intro to the Program

The program was split into several broad categories of tasks: resetting the position, startup, shutdown, motor movement, board reading, and the logic to determine the optimal move.

Resetting the position was an important step considering the motor controls were found to be unreliable during testing prior to the project in MTE 121 assignments. To accommodate for the inaccuracy, the robot zeroed itself during runtime and always did once at the beginning of the program. The startup and shutdown procedures were incredibly important to have their own blocks because they defined the initial position the robot was found in, defining how the robot continued by scanning the board. The shutdown defined how the robot finished each turn.

The regular operation of the robot consisted of scanning the board, picking up, and placing the piece in the optimal place. Motor movement, board reading, and the logic to find the optimal move were each integral part of being able to do this. Motor movement was necessary to both move and activate the claw. Board reading was necessary to interpret the board, and logic was to determine the next piece's placement.

5.2 Task List

Below is the task list determined for the robot demonstration.

Startup

- Program starts by hitting check mark
- Robot moves to a corner to zero all motors

Regular Operation – Expected

- Waits for touchLED to be pressed (for under 3 seconds)
- The motors move the optical sensor to each square on the board to determine whose turn it is (X's or O's) and which pieces are where
- Motors move the claw to the X-stack or O-stack (depending on which turn it is)
- Motors move the claw to the correct piece height (which is determined by the distance sensor)
- Claw extends to grab the piece

- Claw is moved to the position for the optimal move
- Claw retracts to release the piece and is moved to the corner for motors to be reset

Regular Operation – Unexpected

- If the optical sensor scans the board and the game is over, or the position is invalid (such as 3 X's and 1 O) then it makes a noise and returns to the corner to reset position
- If the robot is not at zero at the start of the turn (it has been moved by the player) it resets its position before moving

Shutdown Procedure

- If touchLED pressed for more than 3 seconds in between moves, robot shuts down

A few changes were made from previous task lists. One of the biggest changes was adding far more zeroing steps, such as before and after every turn, because testing determined the robot was very inaccurate without it. Additionally, it was originally five seconds of holding the touchLED to turn the program off, but users were found to be impatient in testing.

5.3 The Functions

The most important function was main, defined at [Appendix A, line 190], because it defines how all other function should be implemented. main was written by Maxim Beauchemin with no parametres and return type integer as standard in C++. The flowchart below, Figure 16 demonstrates the purpose of main, it started up the program by creating an empty array for the board, which is further discussed in 5.4 Data Storage. It reset the position, called the waitOrEndProgram function which determined if it is appropriate to end the program. Lastly, it did all called functions specified in Figure 16.

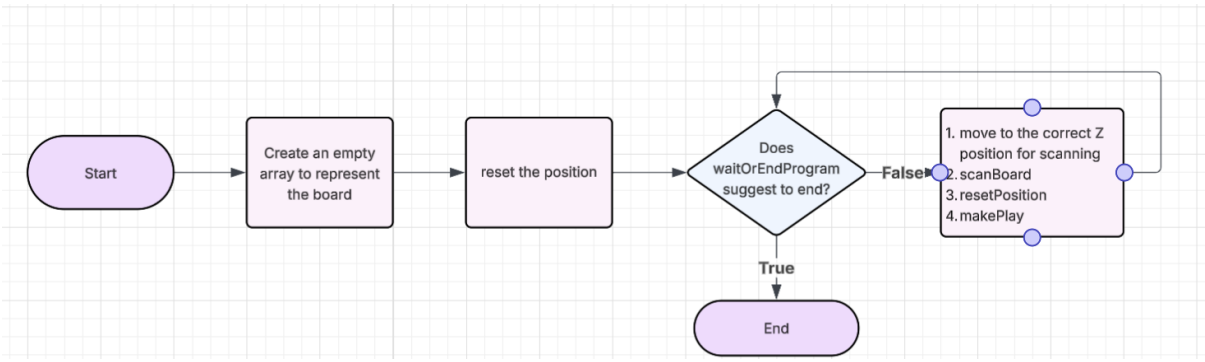


Figure 16. Flowchart for main

The function, configureAllSensors, defined at [Appendix A, line 220] and has no parameters and return type void. It was written by Presley Prince and is a modified version of the function provided by MTE 121. It is considered a trivial function, so no flow chart is provided.

The function, waitOrEndProgram, defined at [Appendix A, line 253], has no parameters and return type bool or Boolean. It was written by Andrew Jang. As discussed above, it determined whether the program should end by returning true if the user holds the TouchLED for a long time and false if it is merely tapped. This functionality is displayed in Figure 17.

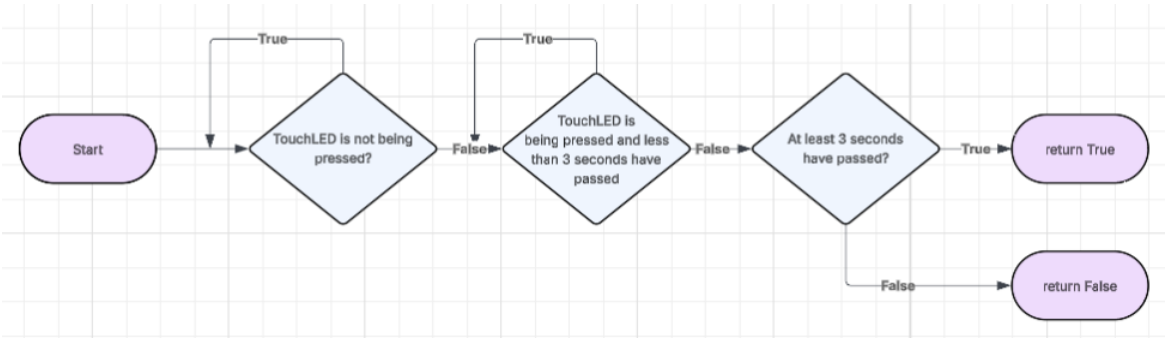


Figure 17. Flowchart for the waitOrEndProgram function.

The pair of function moveToXY and moveToZ, defined at [Appendix A, line 270] and defined at [Appendix A, line 305], moved the claw to a specific x, y or z position respectively. Both were written by Chris Dietrich and accepted appropriate parameters, integers x and y for moveToXY and integer z for moveToZ. Neither function has a return type. The purpose of moveToXY and moveToZ are displayed in Figure 18 and Figure 19 below. They work similarly; the robot moved in the appropriate direction until it reached the end value given by the parameter. Their main difference, that the Z motor first releases the brakes, is explained in 5.7 Significant Problems.

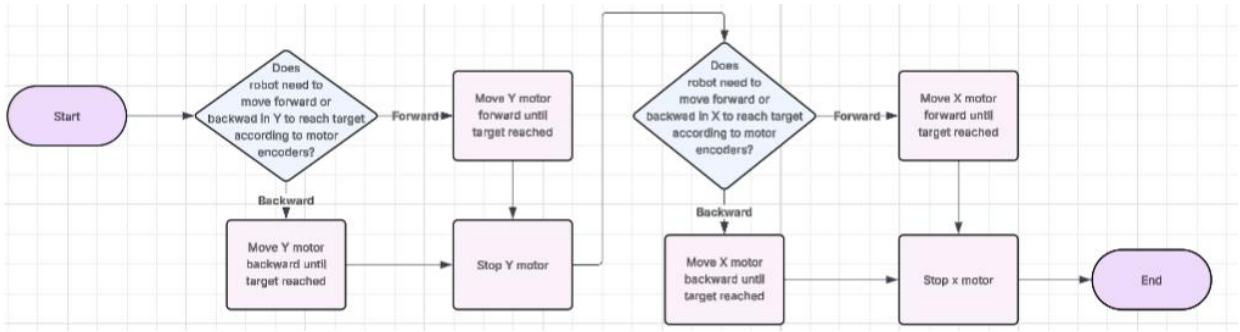


Figure 18. Flowchart for moveToXY.

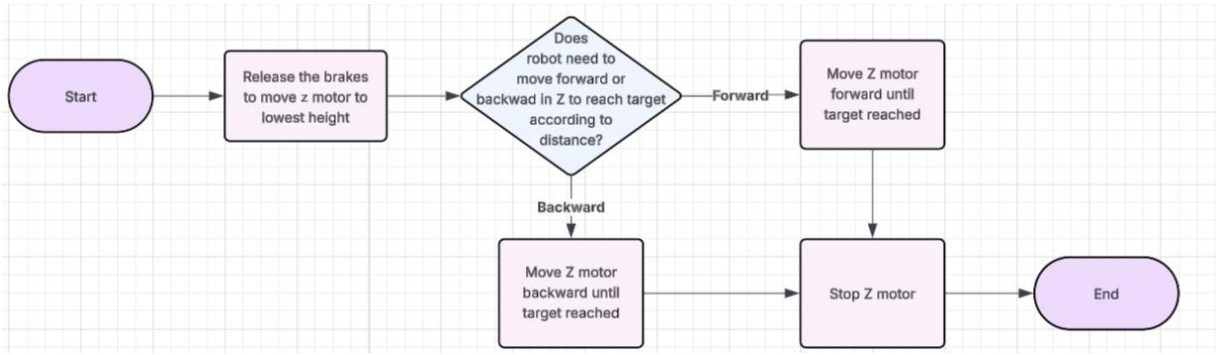


Figure 19. Flowchart for moveToZ

The pair of functions clawOn and clawOff, defined at [Appendix A, line 331], written by Presley Prince control the pins on the claw, the mechanical side which was already described. Neither had any parameters or a return type. The purpose of each function is in the name, to extend the claws, and retract the claws, which picked up or releases an X or O piece. Figure 20 clarifies this functionality, demonstrating that the program either checked if it has been three seconds–something would be wrong, and this corrected it—or if it reached its target.

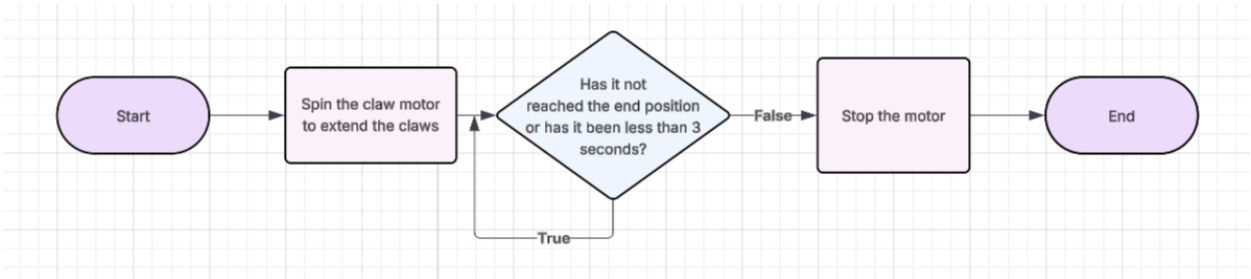


Figure 20. Flowchart for clawOn

The two functions subOptimalSpace and checkWin, defined at [Appendix A, line 359] and [Appendix A, line 527], respectively, were written by Maxim Beauchemin. checkWin accepted the board array and checked through it using a series of conditionals, as seen in Figure 21 to determine if a player has won, returning a char variable which represents the winner. The subOptimalSquare function used these values to predict upcoming moves. Its parameters were the current board array, a character which represented the current turn, and an integer which represented the number of pieces left in the pile. The latter two are reference variables so they could be updated for use in main. As Figure 22

demonstrates, subOptimalSquare created a copy of the board to modify and called checkWin to determine if any move led to an immediate win, block the opponent from an immediate win, or led to a win in two moves.

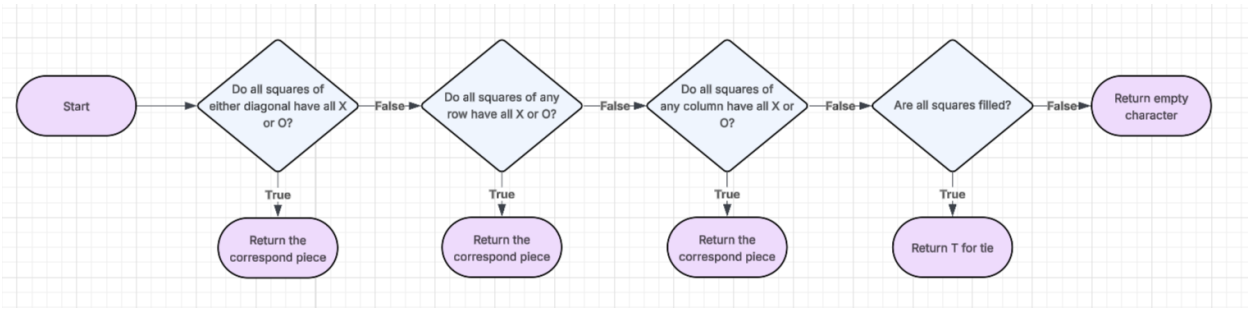


Figure 21. Flowchart for checkWin.

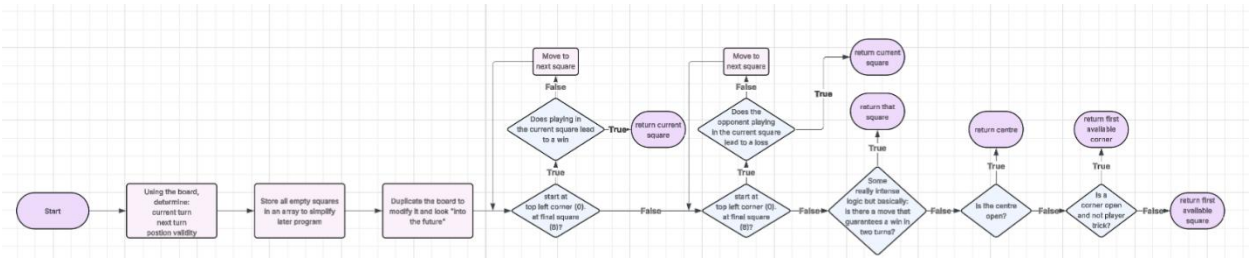


Figure 22. Flowchart for subOptimalSquare

The scanBoard function, defined at [Appendix A, line 564], written by Chris Dietrich, was closely linked to both the board array because it received a pointer to that array as a parameter, and the moveToXY function because it called it once to move to each square of the board before scanning it. As described in the Figure 23 flowchart, it found the lowest value given by the Vex IQ Optical to decrease error. It had no return type.

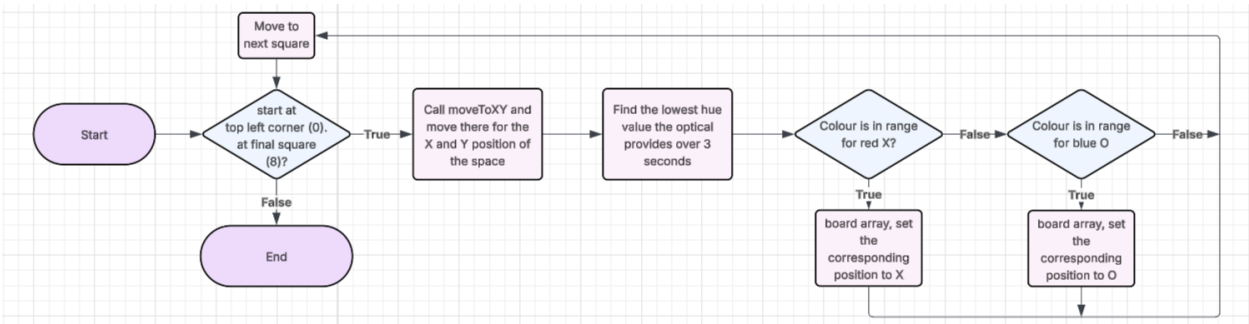


Figure 23. Flowchart for scanBoard

The makePlay function, defined at [Appendix A, line 601], written by Andrew Jang, was closely related to the suboptimalPosition function because its purpose was to interpret parameters: the integer that represents the optimal position, the current turn character and the number of remaining pieces. It then played those moves onto the physical board using a series of function calls described in Figure 24. It had no return type.

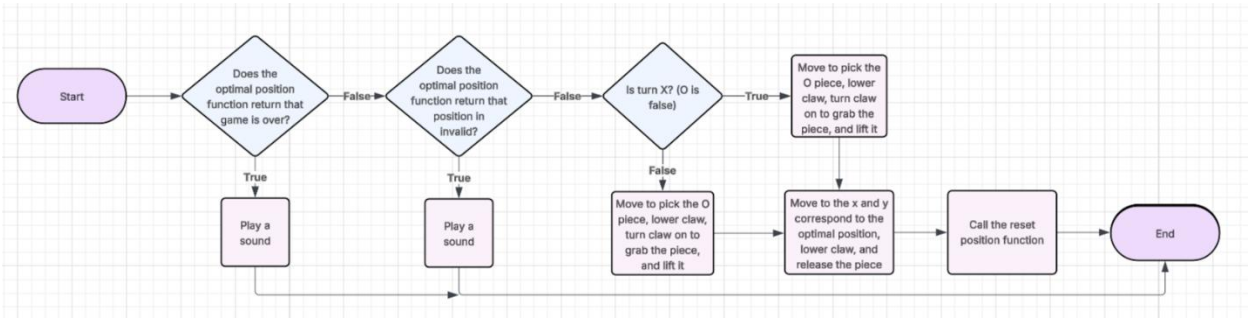


Figure 24. Flowchart for makePlay.

Lastly, but most importantly, the resetPosition function, defined at [Appendix A, line 652], was written by Andrew Jang. Its purpose was to set the claw to predetermined position using a series of timers, as illustrated in Figure 25 and set the position of the motors to zero. This ensured that the claw started in the same position every time the program was run. It had no parameters or return type.

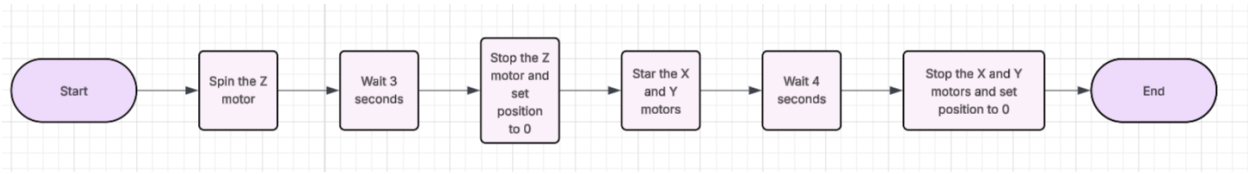


Figure 25. Flowchart for resetPosition.

5.4 Data Storage

Other than the constants, the program only had three variables, but each had an integral role in the program. First was a character variable, turnXO, which represented the piece that the robot should play: 'X' for the X piece and 'O' for the O piece. This was both important in the subOptimalSquare function and the makePlay function to determine which player to find the optimal move for and where

to pick up the piece. When subOptimalSquare was called, it determined the value of turnXO and updated it through pass-by-reference.

Secondly, an integer variable, piecesLeft, represented the number of pieces left in the stack for the current player. It was essential in determining the correct height of the piece which was to be grabbed by the claw.

Lastly, and most importantly, was the 1D character array, board. This array stored all the positions of the piece once scanned by the array with 'X' representing the X piece, 'O' representing the O piece, and the space character representing an empty square. Below, Figure 26 is a graphic representing the index associated with each position of the board.

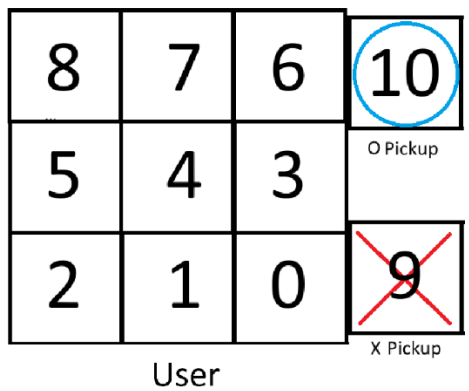


Figure 26. Board representation including array access indices and user location.

Throughout the entire rest of the program these values represented the three states each square could be in which was essential for using the equality comparison.

5.5 Decisions and Trade-offs

Many decisions were made surrounding the robot's program. One of the biggest was whether to have a Tic-Tac-Toe program which always plays the best move or one that a first-year engineer could program alone. According to , the Minmax algorithm consistently outputs the best move, however it uses a recursive algorithm which a first-year could not independently invent. It was decided that a different algorithm would be used: subOptimalSquare written by Maxim Beauchemin, which never lost in testing. However, it struggled with winning because Minmax looks forward to the end of the game while subOptimalSquare only predicts up to two moves ahead.

Another major trade-off involved the resetPosition function. When called, all motors would head straight for a wall and attempt to continue moving past the wall. While ensuring it always reached the reset position, it worsened the user experience through adding a long waiting time. It was decided to maintain the functionality though because the robot could not predict its position without it.

5.6 Testing Procedures

To test the algorithm, a C++ program was developed using the algorithm and basic code to allow for input and output to the console. It was tested against a variety of test cases and positions, specifically targeting a position where it would win in one move, where the user would win in one move, and where it could win in two moves.

To test that the board was properly scanned, the team gave it several boards to scan, especially ensuring it could scan corner squares that it was not fully centered on. It also focused on giving it an empty square surrounded by other pieces of the same color to test if they would be wrongfully scanned.

The major tests involved piece pickup and drop-off were involved with refining the constants which represented different squares and heights on the board. ~~Below, Table 1~~Below, Table 1, contains the values obtained needed to drop off the board. They were additionally required for scanning. The height of the pieces to pick up was additionally tested, and values can be seen in Table 2.

Table 1. A table of all X and Y positions for the motors for each square of the board. The position is given in degrees and obtained from motor encoders through testing. The array indices are visualized in Figure 26.

Square	Array Index	X Position (degrees)	Y Position (degrees)
Bottom Right	0	-190	0
Bottom Middle	1	-190	-100
Bottom Right	2	-190	-240
Centre Left	3	-345	0
Centre Middle	4	-345	-100
Centre Right	5	-345	-240
Top Left	6	-498	0
Top Middle	7	-498	-100
Top Right	8	-498	-240
O Pickup	9	-470	-362
X Pickup	10	-200	-357

Table 2. A table of distance sensor readings corresponding to the number of pieces left in the stack. The values assume that all pieces are left in the stack or on the board. The values were obtained through distance sensor reading and refined in testing.

Pieces Left	O Pickup Distance (mm)	X Pickup Distance (mm)
1	85	85
2	92	88
3	98	96
4	105	103
5	116	120

Lastly, the entire program was integrated together, and it was repeatedly tested by playing against it using similar cases as above.

5.7 Significant Problems

During testing, two major problems sparked. The first problem involved the Vex IQ optical sensor's ability to detect the color of the pieces, which is important for determining what the next move should be. Even one misread square leads to a completely different output, such as an extra X turning a perfectly valid position into an invalid one. While in practice in MTE 121, the optical sensor's light was always turned on because it improved its ability to read colors. However, the optical sensor was much further away from the pieces than the recommended limit, which was a major design flaw, but it had a smart software solution. The testing process identified that when the light was off, the board's black color was either read as black or red, but when the light was on, all the hues suggested purple. The solution was to take the minimum hue over two seconds, which for black, was always zero and for red, was always roughly 300.

The second major problem was again rooted in a mechanical design flaw but was fixed with software. The z-axis movement was very inaccurate because a small rotation caused a large displacement and poor motors worsened the issue. The issue was more prevalent when lowering because it was aided by gravity, but less prevalent when moving upwards because gravity would force the motor to rotate more slowly. By switching the brakes from hold to coast, the motors would first drop, then move upwards to the correct position, but working against gravity to decrease velocity and achieve a more precise result. While a mechanical solution was preferred, the deadline was too soon for a mechanical remodel.

6.0 Verification

6.1 Fulfilled Constraints

The group was logical and creative in addressing most constraints. To address the initial constraint, short wire lengths, the brain was placed in the middle of the aluminum extrusion, where it was most equidistant to all motors and sensors, considering the ratio of the number of small to large wires provided. The maximum distances where the aluminum extrusion and the wheel mechanism could move determined the length and width of the board. However, this created another constraint, the high battery mass located at the center of the board. To account for the skew of the aluminum extrusion when travelling in the X-direction, a guided rail was built to prevent the wheels attached to the aluminum extrusion from straying from its path, as shown in 4.6 Support Wheel Figure 14. The limited number of motors provided, and the amount of weight each motor had to carry, was solved by either adding more connections or duct tape for structural integrity or by using contraptions like these to guide motion. Another contraption can be found in the linear rack system, where a row of corner connectors was placed on either side of the gear to prevent it from slipping.

Low 3D printer resolution was accounted for by adding more than enough clearance in the CAD specifications of the part before 3D printing. Therefore, the claw protrusions had more than enough space to insert themselves into the piece. Furthermore, since the group could not afford as many fasteners as needed, duct tape was the preferred alternative to structural integrity. As for imprecision in sensor measurements, the optical sensor was covered with paper on all sides except one to remove as much light interference as possible. Light levels within the room during demo day and testing were also carefully monitored. On the other hand, the distance sensor was positioned so that the surface that it was reading was consistent, i.e., it was placed over the supporting panel of cardboard instead of the X and O pieces as initially planned. The ridges in VEX IQ beams were resolved by tightly wrapping them with paper. This allowed for mostly consistent smooth motion.

6.2 Failed Constraints

More fasteners and beams could have been added to improve the structural integrity of the robot; however, it was in the best possible shape for the budget that the group had set. Only one real constraint failed to be properly met in this design. Failure in resolving poor motor precision caused the robot, specifically the motor that controls motion vertically, to be incapable of picking up pieces in a lot of the test cases. However, poor motor precision was caused by a lot of factors, such as the weight the

motor was carrying, how long it was in use, and the battery charge. It was an extremely difficult constraint to control. Also, as mentioned before, VEX IQ's battery power levels were depleted by nearly 5% during each test run. As motor strength is directly dependent on the battery charge, this made it almost impossible to calculate the correct number of rotations needed to lower the claw to the right height, given that the battery charge could be at any percentage at any time of operation.

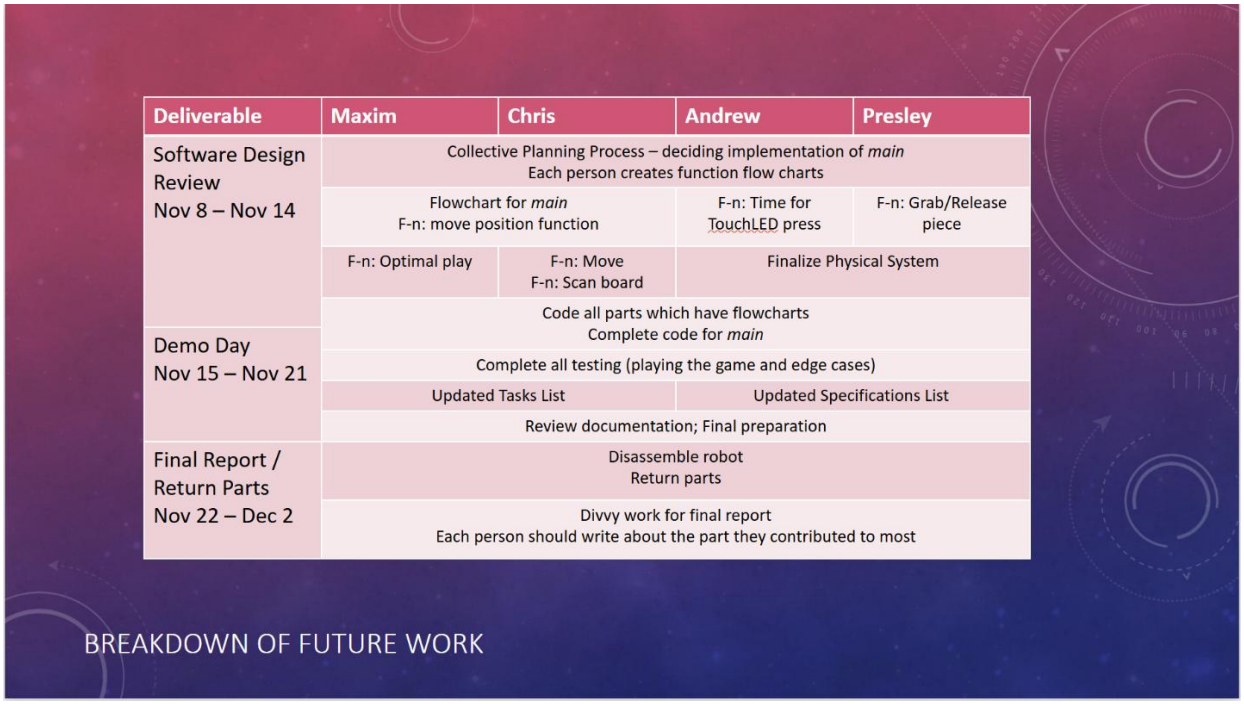
A design update that could resolve this issue is adding gear reduction instead of directly connecting the motor to the gear that rotates to change the height of the claw. A smaller gear could be attached to the motor instead; the smaller gear could bite with the larger gear, allowing for less speed, and therefore, less movement per shaft rotation, which translates to more torque, and therefore, more control over the height of the claw. A more compact alternative to this idea is a spur gearbox that could have offered the same benefits but at a higher cost. This change would allow for a higher acceptable range of values for piece pick-up at each height increment of the stack, reducing its chances of failure.

7.0 Project Plan

The early stages were not as work intensive as the later stages, but each group member was given one week to work independently or with another member on their tasks. This was in preparation for our weekly meetings Monday. Since all group members all had the same class schedule, it was easy to discover that all members had extremely open Mondays after the MTE 121 lecture and before the MATH 115 lectures: roughly four hours. As a result, they became the designated meeting day.

The team decided to split out tasks to work in smaller groups because all four had different schedules and varying work styles, but it allowed them to stay well on track.

The last two weeks leading up to demo day were more together, so each member was responsible for clearing up a large section of time. The robot was kept at Village 1 since most members lived there and it was decided as a group that meant more time for the project and less travel time. Below, Figure 27, shows the project plan following the formal presentation.



Deliverable	Maxim	Chris	Andrew	Presley
Software Design Review Nov 8 – Nov 14	Collective Planning Process – deciding implementation of *main* Each person creates function flow charts			
	Flowchart for *main* F-n: move position function		F-n: Time for TouchLED press	F-n: Grab/Release piece
	F-n: Optimal play	F-n: Move F-n: Scan board	Finalize Physical System	
Demo Day Nov 15 – Nov 21	Code all parts which have flowcharts Complete code for *main*			
	Complete all testing (playing the game and edge cases)			
	Updated Tasks List		Updated Specifications List	
	Review documentation; Final preparation			
Final Report / Return Parts Nov 22 – Dec 2	Disassemble robot Return parts			
	Divvy work for final report Each person should write about the part they contributed to most			

Figure 27. A description of the project plan after the formal presentation. Taken from the formal presentation slide show.

It also demonstrates that a significant portion of the project plan revolved around the near-weekly deliverables. This seemingly good issue became an issue which is explained in 7.3 Project Plan vs Actual Timing.

7.1 Tasks Splitting

For the mechanical half, the plan split the team into two, Maxim and Chris worked on the claw mechanism and the board. Together, they were to create the SolidWorks models for the X and O pieces, all parts of the claw, and the board. Maxim was additionally tasked with learning the 3D printing while Chris was tasked with motion along the Z axis.

Andrew and Presley were to focus more on creating the frame for the robot along with motion for the X and Y axes using a pair of racks and pinions. They were to take the dimensions Maxim gave them for the size of pieces and the board and translated them into frame dimensions. They worked much more on assembly than individual pieces but did additionally design the aluminum extrusion.

For the software half, each member was given roughly two functions to write. The more logically complex functions were given to the more experienced members, Chris and Maxim, but the overall programming time was equally split. The specifics for who was assigned each function is specified in 5.3 The Functions.

7.2 Revisions to Project Plan

Overall, the project plan was created and refined throughout the project to reflect on how the team's work progressed, so it is difficult to define strict revisions to the plan.

However, there were moments when the team was in crisis and a necessary change was made. As seen in Figure 27, the time allocated for testing was extremely short, while simple tasks such as the tasks and specifications lists were given equally long sections. However, as soon as testing started, the team realized that not enough time was allocated for it, especially considering the program was not even finished by the allocated first day of testing, which while being according to plan, was clearly a lapse in judgement. It was agreed from then, as a revision to original plan, that every day that week, time after lectures would be spent working on the project. It was an unfortunate sacrifice, but it was necessary to finish by the day of the demonstration.

Another change to the project plan was to remove the flowcharts stated in Figure 27 because they are not specified in the Software Review deliverable. While they are included in 5.3 The Functions, they were created after the program was written. The reason for this revision was that all the group members found creating flowcharts easier after writing the program rather than before.

7.3 Project Plan vs Actual Timing

For the most part, especially earlier stages of the project, the group timed everything very well with the actual plan. The plan probably held the team back because it meant that they only had to have a certain amount of work completed. Once they were done working, they could wait for the next deadline.

This was especially a problem because the nine deliverables were the foundation for the timing. For example, the first two weeks of the project were focused on assessing the need aspect of the design cycle, but the only project idea was due, which was worth a very small part of the grade and was very easy. At this point, the team was far ahead of schedule, but then merely waited until the next deliverable was due.

The culmination of this issue was most apparent in the week leading up to the project demonstration. Up to this point, the actual timing of the project was very close to the project plan. However, the team far underestimated the amount of testing needed, as is apparent in Figure 27. The team fell far behind, not completing the testing until very late the night before. The main issue with the process used for generating the plan, as the team discovered that night, was that only focusing on deliverables leads to a cram before it is due instead of consistent, much less stressful, work.

8.0 Conclusions

8.1 Successes

The robot successfully overcame many of the challenges it faced. The positioning of components was successfully reworked to accommodate the limited wire lengths. This successful redesign ensured that all components of the robot were operational and secure. The robot was able to move along all three axes, although it struggled with z-axis motion as usage continued. Movement in the xy-plane was accurate enough to successfully scan the board and place pieces. The claw mechanism worked very well as pieces were easily caught and released by the claw. Each piece had a clearance fit with the claw and the board that further improved the robot's accuracy.

The robot consistently made correct decisions for piece placement when faced with a board layout. The shutdown and starting procedures operated as expected. When faced with a completed game or cheating, the robot successfully returned to its starting position and informed the user. From a software perspective, the project met all criteria and constraints.

8.2 Failures

The robot was successful during demonstrations but required further improvements and redesigns to improve its usability. The robot scanned the board and placed pieces in the intended locations during testing but fell short of being able to play full Tic-Tac-Toe games. This was due to motor control and accuracy. The primary challenge was the accuracy and power of the z-axis motor which worsened after extended usage. The motor worked as intended if it was given a two-to-three-hour rest period. However, after roughly half an hour to an hour of usage, the motor became noticeably weaker and could not complete its intended tasks. The motor became unable to lift the claw up and down and required cooldown time before its next use. Aside from this flaw in the physical design, the robot was able to complete the tasks of scanning the board and placing pieces. If the z-axis motion were to be reworked, the robot would have been functionally able to play Tic-Tac-Toe. However, beyond movement in the z-axis, many additional changes would have improved the usability of the robot. These changes are discussed in detail in 9.0 Recommendations.

9.0 Recommendations

9.1 Mechanical Recommendations

As mentioned in 6.2 Failed Constraints, a change to address the imprecision of the motor controlling the height of the claw is to add gear reduction to it, which can either be accomplished by meshing a smaller gear that is connected to the motor to a larger gear, or by adding a spur gearbox for a more compact solution. Less speed and higher torque allow for more control over the height of the claw, since it increases the range of values acceptable for each height at which a piece may lie. This would make it easier to determine the correct motor encoder values during testing.

Another way to improve the robot would be to redesign the board. Each space on the board should be deepened and the angle of the chamfers on the board should be steeper. The increased depth of the squares on the board would decrease how much each piece sticks out above the board. This would have made scanning the board easier. A problem with the original board was that the color sensor would read the color from surrounding pieces. With taller walls between pieces, the surrounding pieces would not be an issue. The steeper chamfers would ensure that the pieces slide into the proper position after being released by the claw. During testing, the robot would sometimes place pieces off-centered enough to stick out above the board Figure 28. This created some problems when scanning the board as claw hangs very close to the board.

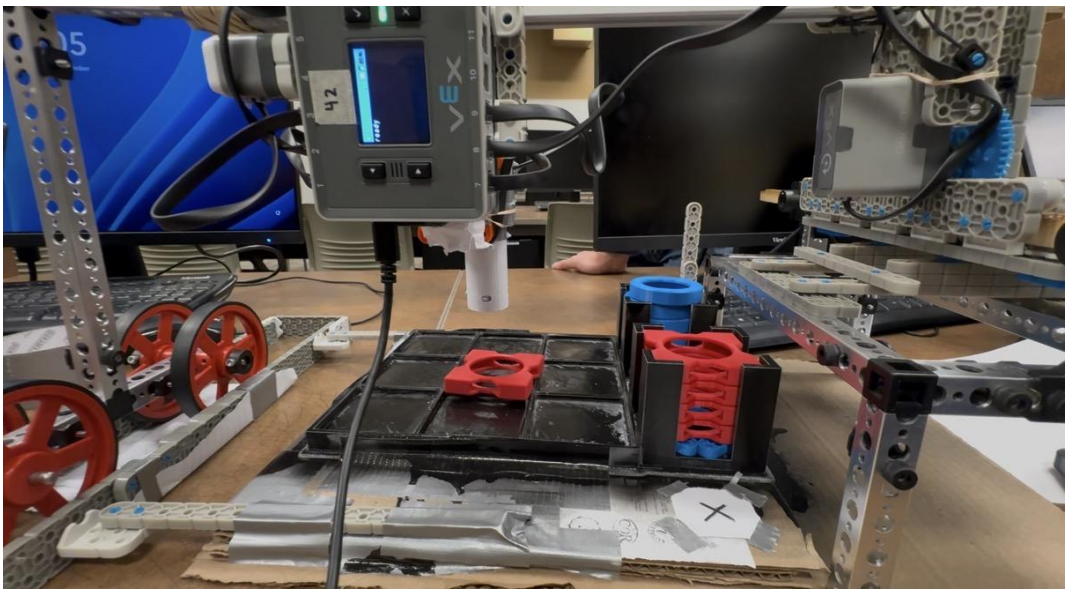


Figure 28. Piece was placed off-centered by robot

Lead screws should also be used to control motion along the three axes. Using lead screws would convert the rotational motion provided by the VEX motors into linear motion as required. Lead screws would have been a more viable alternative to the rack and pinon, wheels, and gears used on the robot. The main benefits of using lead screws are their ability to handle high axial loads and move with greater precision [3]. The increased precision and strength would allow the robot to consistently reach desired heights thereby increasing the functionality of the robot. This redesign would also be less demanding for the z-axis motor. The robot design in this project placed plenty of force on the z-axis motor. The motor would have to constantly work to keep the claw in the raised position. Using a lead screw would allow for the transfer of weight of the claw off the motor.

9.2 Software Recommendations

There are two main flaws with the software that would ideally be changed, but due to the time constraints of the project and limitations of team composition consisting of solely first year students.

Firstly, the algorithm for determining the optimal move, subOptimalMove, relies on a series of only scans two moves into the future. There is potential that the robot could be beaten. However, if the program used a recursive algorithm, like the Minmax algorithm in , it could scan to the end of the game because it would give any potential boards back to itself to keep looking for potential boards.

Secondly, a major improvement to the software surrounds the constants used. If the project had more time, or if the plan was better from the start, more testing time would be available. With this testing time, the team could refine the constants, especially concerning the pick-up, z heights for the pieces, which were often inaccurate even with all improvements. It would be impossible to perfect because the main issue was mechanical 9.1 Mechanical Recommendations, but they could be refined for the initial condition of the motor before it heats up.

References

[1] Vex Robotics, "IQ," Vex Robotics, [Online]. Available: https://www.vexrobotics.com/iq. [Accessed 1 December 2025].

[2] J. Fox, "Tic Tac Toe: Understanding the Minimax Algorithm," never stop building, 13 December 2013. [Online]. Available: https://blogs.cornell.edu/info2040/2022/09/13/56590/. [Accessed 29 November 2025].

[3] G. Leo, “Lead Screws: Mechanics, Design, and Applications,” Madearia, 19 August 2025. [Online]. Available: https://www.madearia.com/blog/lead-screws-mechanics-design-and-applications/. [Accessed 1 December 2025].

Appendix A – C++ Code

```
#pragma region VEXcode Generated Robot Configuration
// Make sure all required headers are included.
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>

#include "vex.h"

using namespace vex;

// Brain should be defined by default
brain Brain;

// START IQ MACROS
#define waitUntil(condition)
  do {
    wait(5, msec);
  } while (!(condition))

#define repeat(iterations)
  for (int iterator = 0; iterator < iterations; iterator++)
// END IQ MACROS

// Robot configuration code.
inertial BrainInertial = inertial();
motor MotorX = motor(PORT8, true);
motor MotorY = motor(PORT6, true);
motor MotorZ = motor(PORT2, false);
motor MotorClaw = motor(PORT9, true);
optical Optical1 = optical(PORT1);
touchled TouchLED3 = touchled(PORT3);
distance Distance7 = distance(PORT7);
```

```
// generating and setting random seed
void initializeRandomSeed(){
  wait(100,msec);
  double xAxis = BrainInertial.acceleration(xaxis) * 1000;
  double yAxis = BrainInertial.acceleration(yaxis) * 1000;
  double zAxis = BrainInertial.acceleration(zaxis) * 1000;
  // Combine these values into a single integer
  int seed = int(
    xAxis + yAxis + zAxis
  );
  // Set the seed
  srand(seed);
}

// Converts a color to a string
const char* convertColorToString(color col) {
  if (col == colorType::red) return "red";
  else if (col == colorType::green) return "green";
  else if (col == colorType::blue) return "blue";
  else if (col == colorType::white) return "white";
  else if (col == colorType::yellow) return "yellow";
  else if (col == colorType::orange) return "orange";
  else if (col == colorType::purple) return "purple";
  else if (col == colorType::cyan) return "cyan";
  else if (col == colorType::black) return "black";
  else if (col == colorType::transparent) return "transparent";
  else if (col == colorType::red_violet) return "red_violet";
  else if (col == colorType::violet) return "violet";
  else if (col == colorType::blue_violet) return "blue_violet";
  else if (col == colorType::blue_green) return "blue_green";
  else if (col == colorType::yellow_green) return "yellow_green";
  else if (col == colorType::yellow_orange) return "yellow_orange";
  else if (col == colorType::red_orange) return "red_orange";
  else if (col == colorType::none) return "none";
```

```
  else return "unknown";
}

// Convert colorType to string
const char* convertColorToString(colorType col) {
  if (col == colorType::red) return "red";
  else if (col == colorType::green) return "green";
  else if (col == colorType::blue) return "blue";
  else if (col == colorType::white) return "white";
  else if (col == colorType::yellow) return "yellow";
  else if (col == colorType::orange) return "orange";
  else if (col == colorType::purple) return "purple";
  else if (col == colorType::cyan) return "cyan";
  else if (col == colorType::black) return "black";
  else if (col == colorType::transparent) return "transparent";
  else if (col == colorType::red_violet) return "red_violet";
  else if (col == colorType::violet) return "violet";
  else if (col == colorType::blue_violet) return "blue_violet";
  else if (col == colorType::blue_green) return "blue_green";
  else if (col == colorType::yellow_green) return "yellow_green";
  else if (col == colorType::yellow_orange) return "yellow_orange";
  else if (col == colorType::red_orange) return "red_orange";
  else if (col == colorType::none) return "none";
  else return "unknown";
}

void vexcodeInit() {

  // Initializing random seed.
  initializeRandomSeed();
}

#pragma endregion VEXcode Generated Robot Configuration

//----------------------------------------------------------------------------
```

```
//
//    Module:       main.cpp
//    Author:       {author}
//    Created:      {date}
//    Description:  IQ project
//
//----------------------------------------------------------------------------

// Include the IQ Library
#include "iq_cpp.h"

// Allows for easier use of the VEX Library
using namespace vex;

// Global constants
const int NUM_OF_COLOURS = 2;
const int NUM_SQUARES = 9;
const int DIMENSIONS = 2;
const int GAME_OVER = -1;
const int INVALID_POSITION = -2;
const double ACTION_WAIT_TIME = 1;

// array access constants
const int BOT_R = 0;
const int BOT_C = 1;
const int BOT_L = 2;
const int MID_R = 3;
const int MID_C = 4;
const int MID_L = 5;
const int TOP_R = 6;
const int TOP_C = 7;
const int TOP_L = 8;
const int X = 0;
const int Y = 1;

```

```
// array of physical board positions in degrees from reset position
// will all be negative because it is motion in reverse
const int BOARD_POSITIONS[NUM_SQUARES + NUM_OF_COLOURS][DIMENSIONS] =
{
  {-190, 0},
  {-190, -100},
  {-190, -240},
  {-345, 0},
  {-345, -100},
  {-345, -240},
  {-498, 0},
  {-498, -100},
  {-498, -240},
  {-470, -362}, // X pickup
  {-200, -357}, // O pickup
};

// function prototypes
// configures all sensors we will use. a modified variation of the given function
void configureAllSensors();

// waits for the TouchLED to be pressed and released and returns 1 if time of
// pressing exceeds 5 seconds
bool waitOrEndProgram();

// move the claw to a specified x, y, and z
void moveToXY(int x, int y);
void moveToZ(int z);

// switches the claw from grabbing to not grabbing a piece or vice versa
void clawOn();
void clawOff();

// returns the optimal position for play given a board
int subOptimalSquare(const char board[], char& turnXO, int& piecesLeft);
```

```
// helper function for optimalSpace
char checkWin(const char board[]);

// scan the board to change the board array to correspond to the physical board
// calls the moveTo function
void scanBoard(char board[]);

// receives the optimal position, picks up a piece, moves to the position, and places
// it down
void makePlay(int optPosition, char turnXO, int piecesLeft);

// resets the position and zereos the motor encoders
void resetPosition();

// main
int main() {
  // Initializing Robot Configuration. DO NOT REMOVE!
  vexcodeInit();
  configureAllSensors();

  const int NUM_SQUARES = 9;
  const int SCAN_Z = 75;

  // represents the board as an array
  char board[NUM_SQUARES] = {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '};
  char currTurn = ' '; // represents who's turn it is
  int piecesLeft = 0; // pieces left in the stack for whoever's turn it is

  // startup procedure
  resetPosition();
  Brain.Screen.print("ready");
  Brain.Screen.newLine();

  // the program should repeat until the waitForTouch returns 1
  while (!waitOrEndProgram())
```

```
  {
    moveToZ(SCAN_Z);
    scanBoard(board);
    resetPosition();
    makePlay(subOptimalSquare(board, currTurn, piecesLeft), currTurn, piecesLeft);
  }
  Brain.programStop();
  return 0;
}

void configureAllSensors()
{
  const int MOTOR_POWER_X = 25;
  const int MOTOR_POWER_Y = 15;
  const int MOTOR_POWER_Z = 60;
  const int MOTOR_POWER_CLAW = 15;

  // calibrate the inertial
  BrainInertial.setHeading(0,degrees);
  BrainInertial.setRotation(0,degrees);
  BrainInertial.calibrate();
  wait(2, seconds);

  // set up motors
  MotorX.setPosition(0, degrees);
  MotorY.setPosition(0, degrees);
  MotorZ.setPosition(0, degrees);
  MotorClaw.setPosition(0, degrees);
  MotorX.setVelocity(MOTOR_POWER_X, percent);
  MotorY.setVelocity(MOTOR_POWER_Y, percent);
  MotorZ.setVelocity(MOTOR_POWER_Z, percent);
  MotorClaw.setVelocity(MOTOR_POWER_CLAW, percent);
  MotorX.setStopping(hold);
  MotorY.setStopping(hold);
  MotorZ.setStopping(hold);
```

```
  MotorClaw.setStopping(brake);

  // screen and optical
  Optical1.setLight(ledState::off);
  Brain.Screen.clearScreen();
  Brain.Screen.setFont(mono15);
}

bool waitOrEndProgram()
{
  const int PROGRAM_END_TIME = 3;

  // waits for TouchLED to be pressed
  while(!TouchLED3.pressing())
  {}

  // if TouchLED is pressed for longer than 3 seconds, the program ends
  Brain.Timer.reset();
  while(TouchLED3.pressing() && Brain.Timer.value() < PROGRAM_END_TIME)
  {}
  if(Brain.Timer.value() >= PROGRAM_END_TIME)
    return true;
  return false;
}

void moveToXY(int x, int y)
{
  // moves backward in Y until reaching a position
  if (MotorY.position(degrees) > y)
  {
    MotorY.spin(reverse);
    while(MotorY.position(degrees) > y)
    {}
  }

```

```
    // moves forward in Y until reaching a position
    else if (MotorY.position(degrees) < y)
    {
      MotorY.spin(forward);
      while(MotorY.position(degrees) < y)
      {}
    }
    MotorY.stop();

    // same for X as for Y
    if (MotorX.position(degrees) > x)
    {
      MotorX.spin(reverse);
      while(MotorX.position(degrees) > x)
      {}
    }
    else if (MotorX.position(degrees) < x)
    {
      MotorX.spin(forward);
      while(MotorX.position(degrees) < x)
      {}
    }
    MotorX.stop();
  }

  void moveToZ(int z)
  {
    // dropping the motor, then lifting it, helps because it is more accurate going up
    // then down
    MotorZ.setStopping(coast);
    MotorZ.stop();
    MotorZ.setStopping(hold);
    wait(ACTION_WAIT_TIME, seconds);

    // uses the distance sensor to determine the height of the robot
```

```
  if (Distance7.objectDistance(mm) < z)
  {
    MotorZ.spin(reverse);
    while(Distance7.objectDistance(mm) < z)
    {}
  }
  // would only move down slightly if it has not dropped to the bottom
  else if (Distance7.objectDistance(mm) > z)
  {
    MotorZ.spin(forward);
    while(Distance7.objectDistance(mm) > z)
    {}
  }
  MotorZ.stop();
}

void clawOn()
{
  const int CLAW_DEGREES_EXTENDED = 70;
  const int STOP_TIME = 3;

  // spins the claw until it is extended or it has spun for too long
  Brain.Timer.reset();
  MotorClaw.spin(forward);
  while(MotorClaw.position(degrees) <= CLAW_DEGREES_EXTENDED
        && Brain.Timer.value() < STOP_TIME)
  {}
  MotorClaw.stop();
}

void clawOff()
{
  const int STOP_TIME = 1;
  const int CLAW_DEGREES_RETRACTED = 1;

```

```
  // spins the claw until it is retracted or it has spun for too long
  Brain.Timer.reset();
  MotorClaw.spin(reverse);
  while(MotorClaw.position(degrees) >= CLAW_DEGREES_RETRACTED
        || Brain.Timer.value()  < STOP_TIME)
  {}
  MotorClaw.stop();
}

int subOptimalSquare(const char board[], char& currTurn, int& piecesLeft)
{
  const int X_PIECES = 5;
  const int O_PIECES = 4;
  int numX = 0, numO = 0;
  char nextTurn = ' ';
  char futureBoard[NUM_SQUARES] = {' '};

  // counts how many of each colour have been played
  for(int i = 0; i < NUM_SQUARES; i++)
  {
    if (board[i] == 'X')
      numX++;
    else if (board[i] == 'O')
      numO++;
  }

  // finds the current turn and number of pieces left for the person in that turn
  if (numX == numO + 1)
  {
    currTurn = 'O';
    piecesLeft = O_PIECES - numO;
  }
  else if (numO == numX)
  {
    currTurn = 'X';
```

```
    piecesLeft = X_PIECES - numX;
  }
  else // checks that the position is valid
    return INVALID_POSITION;

  // checks if the game is over
  if(checkWin(board) != ' ')
    return GAME_OVER;

  // finds the next turn
  if(currTurn == 'X')
    nextTurn = 'O';
  else
    nextTurn = 'X';

  // creates the modifiable board
  for (int i = 0; i < NUM_SQUARES; i++)
    futureBoard[i] = board[i];

  // creates an array with all empty squares
  int emptySquares[NUM_SQUARES] = {0};
  int numEmptySquares = 0;

  // finds all empty squares
  for (int i = 0; i < NUM_SQUARES; i++)
    if (board[i] == ' ')
    {
      emptySquares[numEmptySquares] = i;
      numEmptySquares++;
    }

  // checks if immediate win
  for (int i = 0; i < numEmptySquares; i++)
  {
    // updates the potential future board
```

```
    futureBoard[emptySquares[i]] = currTurn;

    // plays in a square if it is empty and results in an immediate win
    if (checkWin(futureBoard) == currTurn)
      return emptySquares[i];

    // resets the modifiable board
    futureBoard[emptySquares[i]] = board[emptySquares[i]];
  }

  // checks if another piece could be played by the opposite player
  if (numEmptySquares > 1)
    // checks if immediate loss
    for (int i = 0; i < numEmptySquares; i++)
    {
      // updates the potential future board
      futureBoard[emptySquares[i]] = nextTurn;

      // plays in a square if it is empty and results in an immediate loss
      if (checkWin(futureBoard) == nextTurn)
        return emptySquares[i];

      // resets the modifiable board
      futureBoard[emptySquares[i]] = board[emptySquares[i]];
    }


  // very complicated ALERT! proceed with caution (:
  // checks for a guaranteed win in 2 turns
  // must be at least 2 pieces left of robot's type to win in 2 turns
  if (numEmptySquares > 2)
  {
    // runs through all possibilities of robot plays
    for (int i = 0; i < numEmptySquares; i++)
    {
```

```
455         // variable to count the number of ways to win based on what opponent plays
456         int numWin = 0;
457
458         futureBoard[emptySquares[i]] = currTurn; // possible move is made
459
460         // runs through all possibilities of human play based on robot play
461         for (int j = 0; j < numEmptySquares; j++)
462         {
463           if (i != j) // checks that it does not play in the same spot as played before
464           {
465             futureBoard[emptySquares[j]] = nextTurn; // possible move is made
466
467             // tests if based on a possible move in two turns if the robot could
468             //guarantee a win
469             bool foundWin = 0;
470             for (int k = 0; k < numEmptySquares && !foundWin; k++)
471             {
472               // checks that it does not play in the same spot as played before
473               if (i != k && j != k)
474               {
475
476                 futureBoard[emptySquares[k]] = currTurn; // possible move is made
477
478                 // checks if based on a possible move the robot makes, that one of the
479                 // moves that the opponent makes
480                 // leads to an immediate win
481                 if (checkWin(futureBoard) == currTurn)
482                 {
483                   numWin++;
484                   foundWin = 1;
485                 }
486                 // resets the board
487                 futureBoard[emptySquares[k]] = board[emptySquares[k]];
488               }
489             }
```

```
            futureBoard[emptySquares[j]] = board[emptySquares[j]]; // resets the board
          }
        }
        futureBoard[emptySquares[i]] = board[emptySquares[i]]; // resets the board

        // checks that for a possible move the robot makes,
        // all possible moves the opponent makes lead to an immediate win
        if(numWin == numEmptySquares - 1)
            return emptySquares[i];
      }
    }

    // plays in this order if it sees no danger or immediate threat:
    // centre, corner, side
    if (board[MID_C] == ' ')
      return MID_C;

    // a common trick is used by players. conditional checks to prevent the robot
    // losing to the trick
    if (!(board[BOT_C] == ' ' && board[TOP_C] == ' ' && board[MID_L] == ' '
        && board[MID_R] == ' ' && board[MID_C] == 'O'
        && ((board[TOP_L] == 'X' && board[BOT_R] == 'X') || (board[TOP_L] == 'X'
        && board[BOT_R] == 'X'))))
    {
      for (int i = 0; i < numEmptySquares; i++)
        if (emptySquares[i] % 2 == 0) // all corners are even
          return emptySquares[i];
    }

    // if nothing else, just play in the first empty square
    for (int i = 0; i < numEmptySquares; i++)
      return emptySquares[i];

    // code should never reach here, but just in case
    return INVALID_POSITION;
```

```
525     }
526
527     char checkWin(const char board[])
528     {
529       const int NUM_SIDE_SQUARES = 3;
530
531       // Checking two diagonals for wins
532       if (board[MID_C] == board[BOT_R] && board[TOP_L] == board[BOT_R]
533           && board[BOT_R] != ' ')
534         return board[BOT_R];
535       if (board[MID_C] == board[BOT_L] && board[TOP_R] == board[BOT_L]
536           && board[BOT_L] != ' ')
537         return board[BOT_L];
538
539       // Checking rows
540       for (int i = BOT_R; i <= TOP_L; i += NUM_SIDE_SQUARES)
541         if (board[i+MID_C-MID_R] == board[i] && board[i+MID_L-MID_R] == board[i]
542             && board[i] != ' ')
543           return board[i];
544
545       // Checking columns
546       for (int i = BOT_R; i <= BOT_L; i++)
547         if (board[i+NUM_SIDE_SQUARES] == board[i] &&
548             board[i+NUM_SIDE_SQUARES+NUM_SIDE_SQUARES] == board[i] && board[i] != ' ')
549           return board[i];
550
551       // Checking for tie
552       bool tie = 1;
553       for (int i = BOT_R; i <= TOP_L; i++)
554         if (board[i] == ' ')
555           tie = 0;
556
557       if (tie)
558         return 'T';
559       // If nothing, then the game is still going
```

```
  else
  return ' ';
}

void scanBoard(char board[])
{
  const int X_PIECE_HUE_MAX = 358;
  const int X_PIECE_HUE_MIN = 339;
  const int O_PIECE_HUE_MAX = 295;
  const int O_PIECE_HUE_MIN = 200;
  const int NUM_HUE_CHECKS = 20;
  const int OPTICAL_X_OFFSET = 100;
  const double SCAN_WAIT_TIME = 0.1;

  // loops through each position of the board
  for (int i = 0; i < NUM_SQUARES; i++)
  {
    // moves to a sqsuare
    moveToXY(BOARD_POSITIONS[i][X] +  OPTICAL_X_OFFSET, BOARD_POSITIONS[i][Y]);

    // scans the colour 20 times. The optical is inconsistent, but the lowest value
    // it finds
    // is more consistent, so that is what is used
    int pieceHue = Optical1.hue();
    for (int i = 0; i < NUM_HUE_CHECKS; i++)
    {
      if (Optical1.hue() < pieceHue)
        pieceHue = Optical1.hue();
      wait(SCAN_WAIT_TIME, seconds);
    }

    // checks what the hue represents as a board piece
    if (pieceHue < X_PIECE_HUE_MAX && pieceHue > X_PIECE_HUE_MIN)
      board[i] = 'X';
    else if(pieceHue < O_PIECE_HUE_MAX && pieceHue > O_PIECE_HUE_MIN)
```

```
      board[i] = 'O';
    else
      board[i] = ' ';
  }
}

void makePlay(int optPosition, char turnXO, int piecesLeft)
{
  // array access constants for pickup positions
  const int X_PICKUP = 9;
  const int O_PICKUP = 10;
  const int MAX_NUM_PIECES = 5;
  // distance sensor heights in mm based on
  const int PICKUP_X_Z[MAX_NUM_PIECES] = {85, 88, 96, 103, 120};
  const int PICKUP_O_Z[MAX_NUM_PIECES] = {85, 92, 78, 105, 116};

  // if the position is invalid or the game is over, it plays a sound and stops
  if (optPosition == GAME_OVER)
  {
    Brain.playSound(powerDown);
    return;
  }
  else if (optPosition == INVALID_POSITION)
  {
    Brain.playSound(wrench);
    return;
  }

  // moves to pickup up the pieces, lowers itself, grabs the piece, and lifts back
  // into the air
  if (turnXO == 'X')
    moveToXY(BOARD_POSITIONS[X_PICKUP][X], BOARD_POSITIONS[X_PICKUP][Y]);
  else
    moveToXY(BOARD_POSITIONS[O_PICKUP][X], BOARD_POSITIONS[O_PICKUP][Y]);

```

```
630     // ensures at least one piece is left in the stack
631
632     // Grabs the piece and lifts it
633     if (turnXO == 'X')
634       moveToZ(PICKUP_X_Z[piecesLeft - 1]);
635     else if (turnXO == 'O')
636       moveToZ(PICKUP_O_Z[piecesLeft]);
637     wait(ACTION_WAIT_TIME, seconds);
638     clawOn();
639     wait(ACTION_WAIT_TIME, seconds);
640
641     // moves to the optimal square and releases the pieces
642     resetPosition();
643     moveToXY(BOARD_POSITIONS[optPosition][X], BOARD_POSITIONS[optPosition][Y]);
644     MotorZ.setStopping(coast);
645     MotorZ.stop();
646     MotorZ.setStopping(hold);
647     wait(ACTION_WAIT_TIME, seconds);
648     clawOff();
649     resetPosition();
650   }
651
652   void resetPosition()
653   {
654     const int XY_RESET_TIME = 4;
655     const int Z_RESET_TIME = 3;
656
657     // moves the z motor as high as possible
658     MotorZ.spin(reverse);
659     wait(Z_RESET_TIME, seconds);
660     MotorZ.stop();
661     MotorZ.setPosition(0, degrees);
662
663     // moves the x and y motors to the corner
664     MotorX.spin(forward);
```

```
    MotorY.spin(forward);
    wait(XY_RESET_TIME, seconds);
    MotorX.stop();
    MotorY.stop();
    MotorX.setPosition(0, degrees);
    MotorY.setPosition(0, degrees);
  }
```